

AI-Powered RAG System (Document Assistant)

Full System Architecture & Application Flow

Technology Stack

Framework

- .NET 8 Web API
- ASP.NET Core
- Semantic Kernel
- Entity Framework Core
- SignalR (Realtime Progress)
- MediatR (CQRS)
- FluentValidation
- Serilog

AI

- OpenAI GPT-4.1 / GPT-4o
- Google Gemini 2.x
- Semantic Kernel

Embedding Models

- text-embedding-3-small
- text-embedding-3-large
- Gemini Embeddings

Vector Database

- Pinecone
- Milvus
- Qdrant (Optional Local)

Database

- PostgreSQL

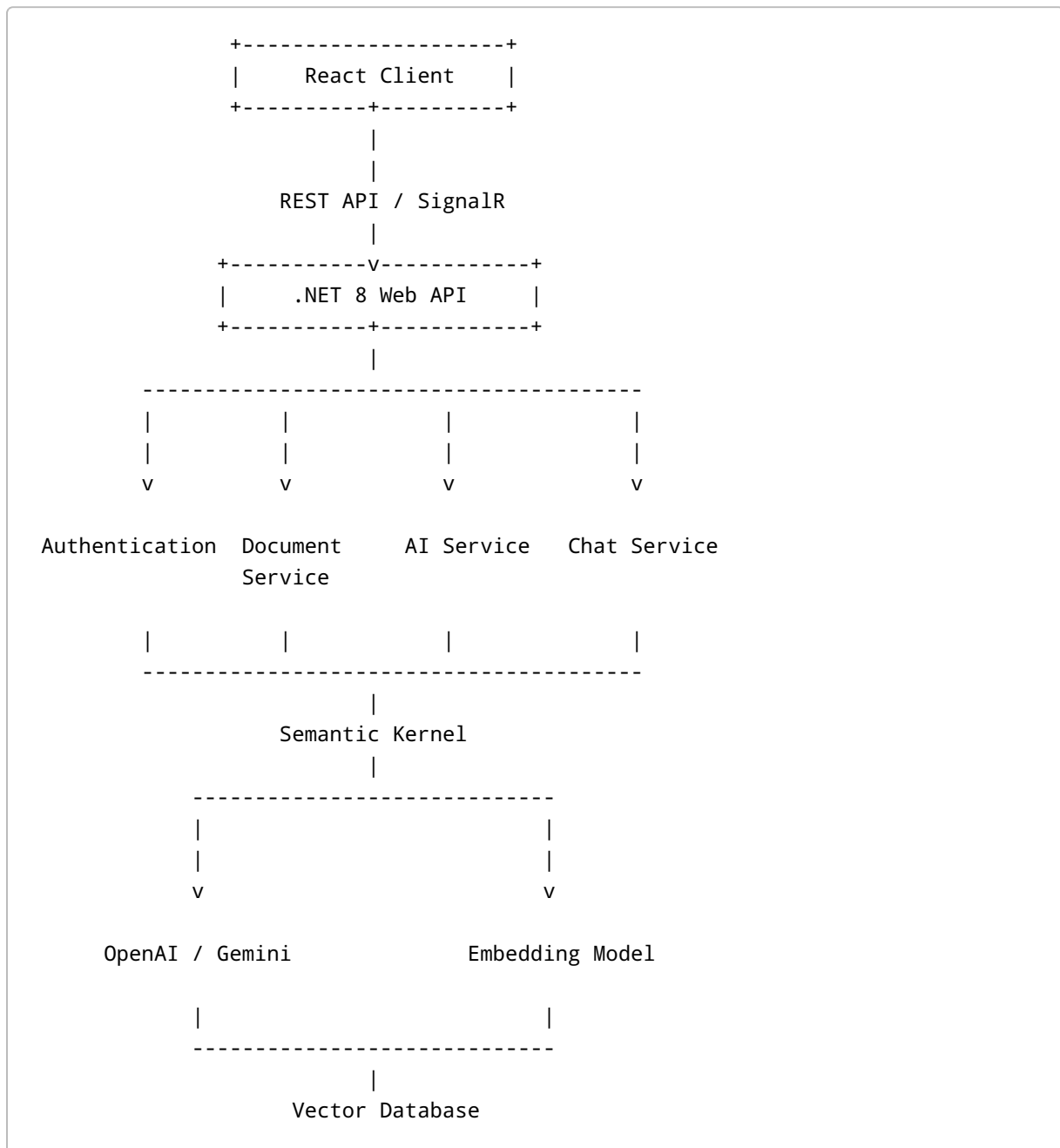
Storage

- Local Storage
- Azure Blob Storage
- AWS S3
- MinIO

Frontend

- React
- Next.js
- Tailwind CSS
- Shadcn UI

Overall Architecture



(Pinecone / Milvus)



PostgreSQL Database

High Level Flow

User



Login



Dashboard



Upload PDF / DOCX



Extract Text



Split into Chunks



Generate Embeddings



Store Embeddings



User asks Question



Convert Question → Embedding

↓

Similarity Search

↓

Top Relevant Chunks

↓

Semantic Kernel Prompt

↓

OpenAI/Gemini

↓

Answer

↓

Display References

Folder Structure

```
src
```

```
|— DocumentAssistant.API
|
|— DocumentAssistant.Application
|
|— DocumentAssistant.Domain
|
|— DocumentAssistant.Infrastructure
|
|— DocumentAssistant.Persistence
|
|— DocumentAssistant.SemanticKernel
|
|— DocumentAssistant.VectorStore
|
```

```
├── DocumentAssistant.Shared
└── DocumentAssistant.Tests
```

Clean Architecture

Presentation

↓

Application

↓

Domain

↓

Infrastructure

↓

External Services

Modules

Authentication

- Register
 - Login
 - JWT
 - Refresh Token
 - Roles
 - Permissions
-

User Module

Create User

Update User

Delete User

Profile

Settings

Document Module

Upload

Delete

Rename

Download

List Documents

View Metadata

View Pages

AI Module

Embedding Generation

Prompt Builder

Semantic Kernel Plugins

Context Window

Memory

Prompt Templates

Response Formatter

Vector Module

Store Embeddings

Delete Embeddings

Search Similar Chunks

Metadata Filter

Namespace Support

Document Processing Pipeline

PDF

↓

Read File

↓

Extract Text

↓

Clean Text

↓

Remove Empty Spaces

↓

Split Paragraphs

↓

Chunking

↓

Overlap

↓

Embedding

↓

Vector Database

Chunk Strategy

Chunk Size

500 Tokens

Overlap

100 Tokens

Example

Chunk 1

Page 1

Words 1-500

Chunk 2

Words 401-900

Chunk 3

Words 801-1300

Database Tables

Users

Id
Name
Email
PasswordHash
CreatedAt

Documents

Id
UserId
Name
FileType
StoragePath
Status
CreatedAt

Chunks

Id
DocumentId
ChunkIndex
Text

EmbeddingId

PageNumber

Conversations

Id

UserId

Title

CreatedAt

Messages

Id

ConversationId

Role

Content

CreatedAt

Pinecone Structure

Namespace

UserId

↓

DocumentId

↓

ChunkId

↓

Embedding

Metadata

UserId

DocumentId

Page

Chunk

Filename

CreatedDate

Semantic Kernel Flow

Question

↓

Generate Embedding

↓

Vector Search

↓

Top 5 Chunks

↓

Build Prompt

↓

OpenAI

↓

Answer

↓

Citation

Prompt Template

You are an AI assistant.

Use only the provided context.

If the answer is unavailable, say:

"I couldn't find that information in the uploaded documents."

Context

{{Context}}

Question

{{Question}}

Chat Flow

Ask Question

↓

Embedding

↓

Pinecone Search

↓

Top 5 Results

↓

Semantic Kernel

↓

LLM

↓

Streaming Answer

↓

Save Chat

API Endpoints

Authentication

POST /api/auth/register

POST /api/auth/login

POST /api/auth/refresh

Documents

POST /api/documents/upload

GET /api/documents

GET /api/documents/{id}

DELETE /api/documents/{id}

Embeddings

POST /api/embeddings/create

DELETE /api/embeddings/{documentId}

Chat

POST /api/chat/ask

GET /api/chat/history

GET /api/chat/{conversationId}

Admin

GET /api/users

GET /api/statistics

Security

- JWT Authentication
- Refresh Tokens
- HTTPS
- File Validation
- Virus Scan
- Rate Limiting
- API Keys in Secret Manager
- Prompt Injection Protection
- SQL Injection Protection
- XSS Protection

Logging

Serilog

Request Logs

Error Logs

AI Response Time

Embedding Time

Upload Time

Token Usage

Caching

Redis

Embedding Cache

Prompt Cache

User Cache

Document Cache

Background Jobs

Upload

↓

Queue

↓

Extract Text

↓

Generate Embeddings

↓

Store Vectors

↓

Notify User

Sequence Diagram

User

↓

Upload File

↓

API

↓

Storage

↓

Text Extraction

↓

Chunking

↓

Embedding

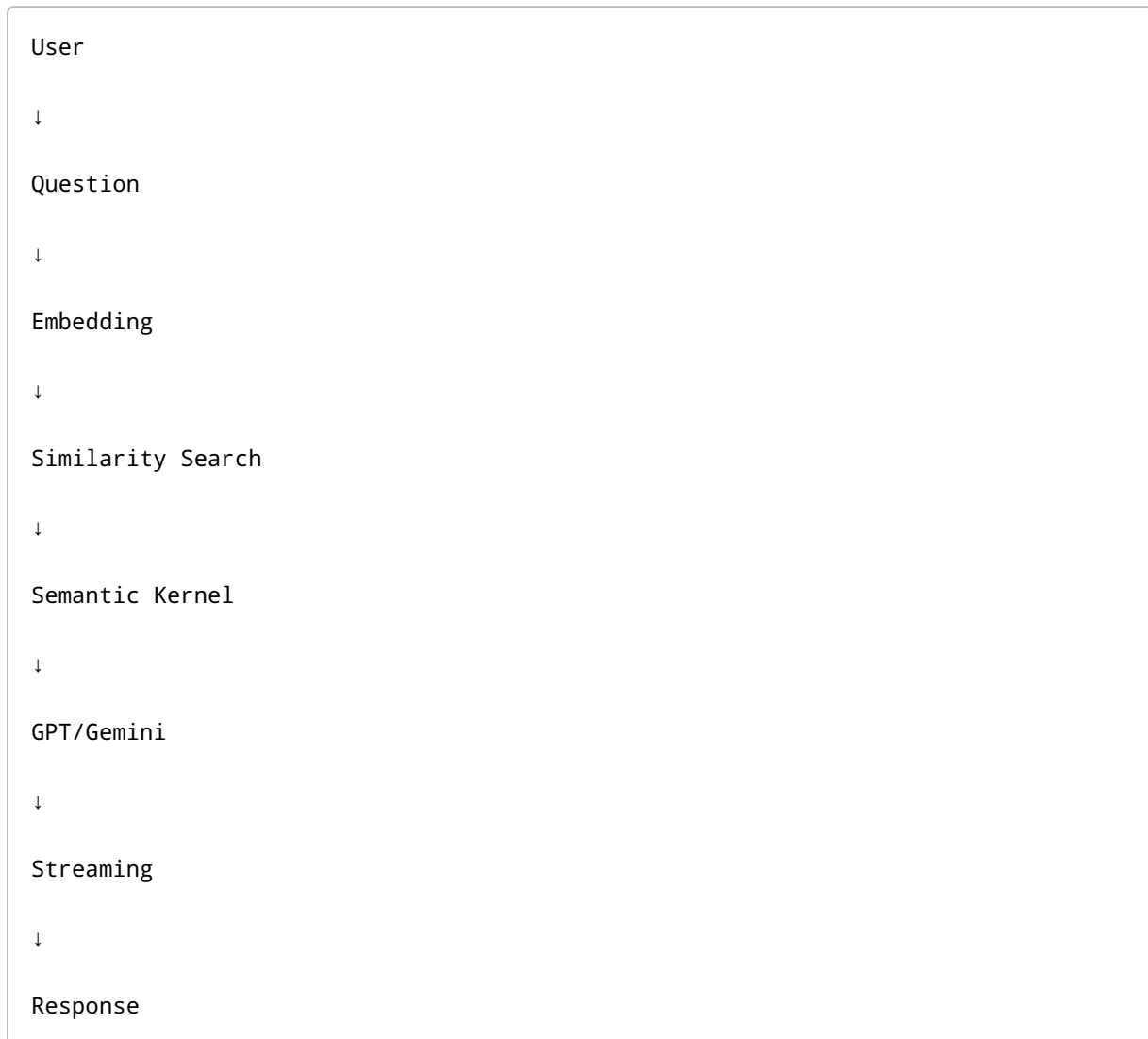
↓

Pinecone

↓

Completed

Chat Sequence



AI Features

- RAG Search
- Semantic Search
- Conversation Memory
- Multi-turn Chat
- Citation Support
- Source References
- Confidence Score
- Streaming Responses
- Multi-document Search

- Document Summarization
 - Keyword Search
 - Hybrid Search
 - OCR Support
 - Markdown Output
 - PDF Export
 - Prompt Templates
 - AI Plugins
 - Tool Calling
-

Future Features

- Image Understanding
 - OCR for Scanned PDFs
 - Voice Chat
 - Speech-to-Text
 - Multi-language Search
 - Agentic AI Workflows
 - Workflow Automation
 - Email Document Analysis
 - Knowledge Graph Integration
 - Microsoft Teams Bot
 - Slack Bot
 - WhatsApp Bot
 - Mobile Application
-

Deployment Architecture

React Frontend

↓

NGINX

↓

.NET 8 API

↓

Redis



Recommended Tech Stack

Layer	Technology
Frontend	React + Next.js + Tailwind CSS
Backend	.NET 8 Web API
AI Orchestration	Semantic Kernel
LLM	OpenAI GPT-4.1 / GPT-4o or Gemini 2.x
Embeddings	text-embedding-3-small / Gemini Embeddings
Vector Database	Pinecone (Cloud) or Milvus (Self-hosted)
Database	PostgreSQL
Cache	Redis
File Storage	Azure Blob Storage / AWS S3 / MinIO
Authentication	JWT + Refresh Tokens
Logging	Serilog
Background Jobs	Hangfire
Deployment	Docker + Kubernetes + NGINX

End-to-End Request Lifecycle

1. User signs in
2. Uploads one or more PDF/DOCX files
3. Files are stored in object storage
4. Background worker extracts text
5. Text is cleaned and chunked
6. Embeddings are generated using OpenAI/Gemini
7. Embeddings and metadata are stored in Pinecone/Milvus
8. User asks a question
9. Query embedding is generated
10. Vector similarity search retrieves the most relevant chunks
11. Semantic Kernel constructs the prompt with retrieved context
12. LLM generates an answer using only retrieved knowledge
13. Response streams to the UI with cited document/page references
14. Conversation history is persisted for future context

Recommended Development Phases

Phase 1

- Authentication
- User Management
- File Upload
- PostgreSQL Setup

Phase 2

- PDF/DOCX Parsing
- Chunking
- Embedding Generation
- Pinecone Integration

Phase 3

- Semantic Kernel Integration
- RAG Pipeline
- Chat Interface
- Streaming Responses

Phase 4

- Conversation History
- Citations

- Redis Cache
- Hangfire Background Jobs

Phase 5

- OCR
- Multi-document Search
- Admin Dashboard
- Docker & Kubernetes Deployment
- Monitoring and Performance Optimization

This architecture follows modern enterprise practices (Clean Architecture + CQRS + Semantic Kernel + Retrieval-Augmented Generation) and is designed to scale from a single-user proof of concept to a production-ready, cloud-native document assistant.